

Relatório Parcial – Esteira Separadora

Sistemas Operacionais Embarcados (SOE) – 2026.1

Autores: Paulo Caleb Fernandes da Silva, Felipe de Castro

1. Descrição do Hardware Utilizado

ATENÇÃO: Documentos auxiliares podem ser encontrados neste repositório no caminho: `../ESTEIRA/ELETRÔNICA/`

O sistema é composto por uma arquitetura híbrida que divide as responsabilidades entre um microprocessador de alto nível e um microcontrolador dedicado ao controle dos dispositivos físicos da esteira.

Raspberry Pi 3B atua como o cérebro central do sistema, sendo responsável por executar os algoritmos de visão computacional para leitura do QR code e classificação de objetos. Uma webcam USB conectada ao Raspberry Pi captura as imagens dos objetos sobre a esteira, permitindo que o processamento de alto nível determine o destino de cada item. As decisões de roteamento são então transmitidas ao microcontrolador via comunicação serial UART.

STM32G070 é o microcontrolador responsável pelo controle direto dos atuadores e pela leitura dos sensores. Opera a uma frequência de CPU de 64 MHz e gerencia os seguintes periféricos:

- **Timer 15 (TIM15):** Gera o sinal PWM de frequência fixa (200 Hz, duty cycle de 50%) utilizado para modular os lasers dos sensores.
- **Timer 3 (TIM3):** Configurado em modo Input Capture com DMA, realiza a medição de frequência dos quatro sensores laser simultaneamente.
- **Timer 16 (TIM16):** Gera o sinal PWM de 50 Hz com duty cycle variável para o controle do servo motor.
- **Timer 6 (TIM6):** Opera a 1 MHz e dispara interrupções periódicas para o controle preciso dos pulsos enviados ao driver do motor de passo.
- **USART2:** Configurada a 115200 bps, 8N1, para comunicação com o Raspberry Pi 3B.

Sensores laser funcionam a partir de um princípio de detecção síncrona por ausência de portadora. Um laser modula seu feixe na mesma frequência gerada pelo TIM15. O feixe incide sobre um LDR que, após condicionamento de sinal, entrega ao STM32 um sinal digital pulsado na faixa de 0 a 3,3 V com frequência idêntica à portadora (200 Hz). Quando um objeto interrompe o feixe, a ausência do sinal pulsado é interpretada pelo firmware como presença de objeto em frente ao sensor.

Motor de passo com driver A4988: O tapete da esteira é acionado por um motor de passo controlado pelo driver A4988. Os pinos STEP, DIR e SLEEP são utilizados respectivamente para: avançar um passo, definir a direção de rotação e habilitar/desabilitar o driver (liberando ou travando o eixo).

Servo motor ES08MA: Responsável pelo acionamento mecânico da cancela que desvia os objetos para a Rota B. É controlado por PWM de 50 Hz, com largura de pulso variando entre 600 µs (0°) e 2300 µs (180°), mapeada diretamente a partir do ângulo desejado pelo firmware.

Flash (luminária): Controlado diretamente por uma GPIO dedicada do STM32G070, sendo acionado durante o estado de classificação para iluminar o objeto e auxiliar a câmera do Raspberry Pi na captura de imagem.

2. Descrição do Software Desenvolvido

O firmware foi desenvolvido em linguagem C utilizando os drivers HAL da STMicroelectronics para a família STM32, com o STM32CubeIDE como ambiente de desenvolvimento.

2.1 Máquina de Estados Finitos (FSM)

O núcleo do firmware é uma Máquina de Estados Finitos (FSM) ainda a ser implementada no sistema;

2.2 Protocolo de Comunicação UART

A comunicação entre o STM32G070 e o Raspberry Pi 3B segue um protocolo proprietário desenvolvido para este projeto. O recebimento é feito por interrupção (`HAL_UART_Receive_IT`), garantindo que nenhum byte seja perdido durante a execução da FSM. O quadro de comunicação é organizado da seguinte forma:

Estrutura do quadro:

Direção	Formato
Recepção (RX)	[START] [CMD] [DATA]
Transmissão positiva (TX)	[CMD_OK] [CMD_RECEBIDO]
Transmissão negativa (TX)	[CMD_ERR]

Onde:

| [START] = 0xAA | [CMD_OK] = 0x90 |
[CMD_ERR] = 0x91 |

Handshake de inicialização:

Byte	Valor	Direção	Descrição
SYS_RDY_MSG	0x10	RX	Raspberry Pi sinaliza que está pronto
SYS_INIT_MSG	0x01	TX	STM32 confirma inicialização do sistema

Mensagens da FSM:

Byte	Valor	Direção	Descrição
OBJ_DETECTED_MSG	0xA0	TX	Objeto detectado na esteira
CLSS_REQUEST_MSG	0xC0	TX	Solicitação de classificação ao Raspberry Pi
ROUTE_A_RECEIVE_MSG	0xDA	RX	Raspberry Pi ordena roteamento para Rota A
ROUTE_B_RECEIVE_MSG	0xDB	RX	Raspberry Pi ordena roteamento para Rota B
ROUTE_A_FWRDNG_MSG	0xFA	TX	Confirmação de encaminhamento para Rota A
ROUTE_B_FWRDNG_MSG	0xFB	TX	Confirmação de encaminhamento para Rota B
ROUTE_A_SCCSS_DLVRY_MSG	0xBA	TX	Entrega concluída na Rota A
ROUTE_B_SCCSS_DLVRY_MSG	0xBB	TX	Entrega concluída na Rota B

Mensagens de controle assíncrono (RX):

Byte	Valor	Descrição
LIGHT_EN_MSG	0xE1	Ativa a luminária
LIGHT_DISABLE_MSG	0xD1	Desativa a luminária
GATE_OPEN_MSG	0xE2	Abre a cancela
GATE_CLOSE_MSG	0xD2	Fecha a cancela
STPR_EN_MSG	0xE3	Ativa o motor de passo (eixo travado)
STPR_DISABLE_MSG	0xD3	Desativa o motor de passo (eixo livre)
SET_STPR_FORWARD_MSG	0xE4	Define rotação do motor para frente
SET_STPR_BACKWARD_MSG	0xD4	Define rotação do motor para trás

Byte	Valor	Descrição
SET_STPR_TGT_STPS_MSG	0xE5	Define quantidade de passos alvo

2.3 Rotina de Inicialização

Antes de entrar no loop principal, o firmware executa uma rotina de inicialização que bloqueia a execução até duas condições serem satisfeitas simultaneamente: todos os quatro sensores laser devem estar operando na frequência esperada de 200 Hz, e o Raspberry Pi deve enviar o byte de handshake `SYS_RDY_MSG` (0x10). Durante essa espera, um LED de depuração pisca rápido (100 ms) se os sensores ainda não estiverem prontos, ou devagar (500 ms) quando os sensores estão operacionais e o sistema aguarda apenas o handshake do Raspberry Pi.

2.4 Controle do Motor de Passo

O controle do motor de passo é realizado inteiramente por interrupção do TIM6. A cada disparo, a ISR verifica se há passos pendentes em `targetStepps` e realiza um toggle no pino STEP, gerando os pulsos necessários ao driver A4988. A velocidade do motor é ajustada modificando o registrador ARR do TIM6 através da função `stepperSetSpeed()`, que limita a operação entre `STEPPER_MIN_VEL` (300 passos/s) e `STEPPER_MAX_VEL` (420 passos/s).

3. Resultados Obtidos nos Testes

Os testes foram realizados com o hardware físico da esteira, validando individualmente cada subsistema do projeto.

Sensores laser: Os quatro sensores operam conforme especificado. A lógica de detecção síncrona por ausência de portadora foi validada com sucesso – ao interromper o feixe, as flags internas (`SENS1_flag` a `SENS4_flag`) são acionadas corretamente, permitindo as transições de estado da FSM.

Flash (luminária): O controle via GPIO dedicada funciona corretamente. O flash é ativado e desativado em resposta aos comandos assíncronos recebidos via UART, sem instabilidades observadas.

Motor de passo: O motor opera de forma estável em frequências mais baixas de acionamento. Os testes indicaram que o controle de velocidade por alteração do ARR do TIM6 funciona, porém com uma faixa operacional limitada – comportamento esperado dadas as características físicas do motor de passo (torque versus velocidade). Em razão disso, a velocidade de operação será mantida fixa no valor mínimo configurado (`STEPPER_MIN_VEL` = 300 passos/s), que corresponde a um período de

interrupção do TIM6 de aproximadamente 1,67 ms, calculado com base na configuração de prescaler de 63 e clock de 64 MHz.

Servo motor: O servo motor ES08MA responde corretamente a qualquer valor de angulação dentro da faixa suportada (0° a 180°). Para a aplicação na esteira, os ângulos definidos no firmware – 0° para cancela fechada (Rota A) e 45° para cancela aberta (Rota B) – foram validados e apresentaram o posicionamento mecânico desejado.